

F-260701-DIS-420-FPGA_EXPLEO

TP02 Compteurs

Question 1

L'horloge du système est fixée à 100MHz. Combien de période faut-il compter pour attendre 2 secondes ? Combien de bits faut-il au minimum pour représenter cette valeur ?

La période d'une horloge de fréquence $f = 100 \text{ MHz}$ est 10 ns: $T = \frac{1}{f} = 10^{-8} \text{ s}$

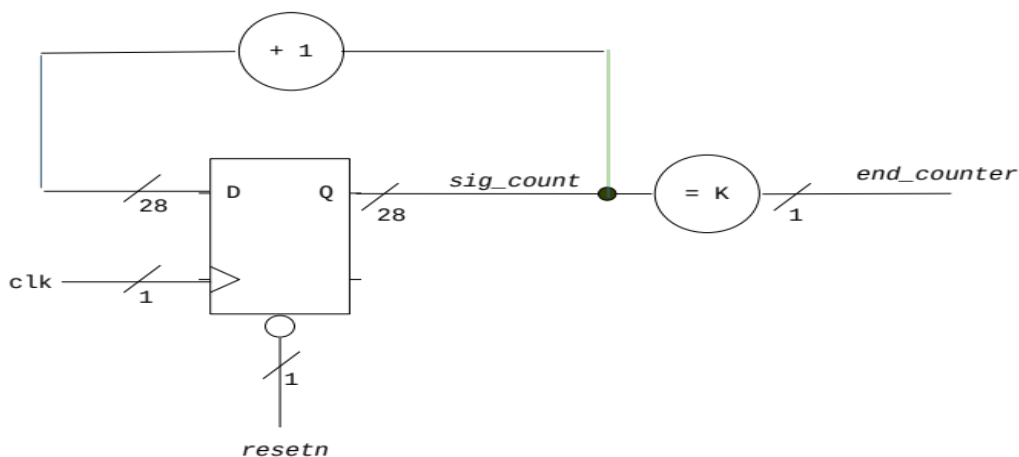
À un intervalle de temps $\Delta t = 2 \text{ s}$ correspond donc $K = \frac{\Delta t}{T} = 2 \cdot 10^8$ périodes¹.

$N = \lceil \log_2(K) \rceil$ bits sont *a priori* nécessaires pour représenter K , soit 28 bits.

Rem: N bits permettant de représenter les entiers non signés dans l'intervalle $[0, 2^N - 1]$, on vérifie également $K < 2^N$.

Question 2

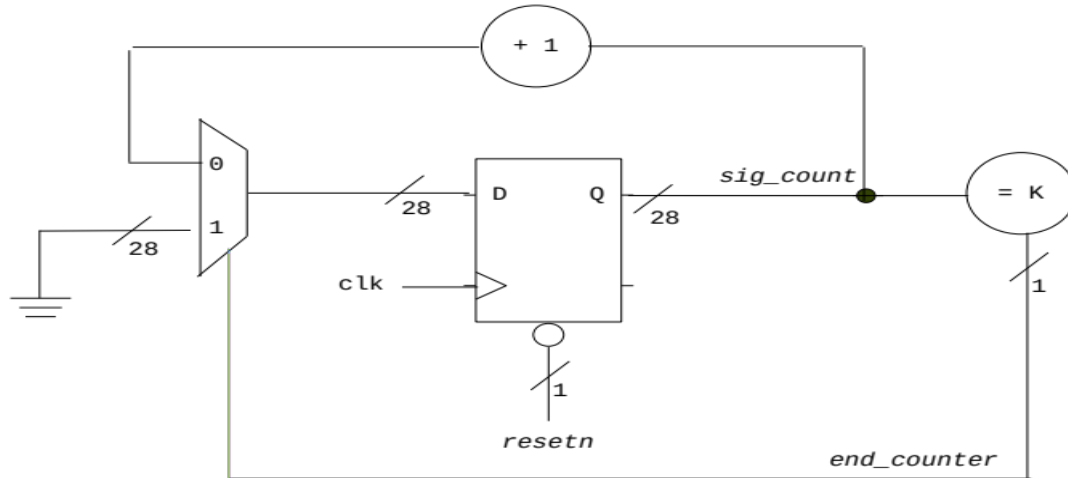
Dessinez le schéma RTL de ce compteur. Si le compteur atteint la valeur calculée précédemment, un signal `end_counter` passe à 1, sinon `end_counter` vaut 0. N'oubliez pas de mettre sur chaque signal son nombre de bits. Commencez par réaliser une boucle d'incrémentation : +1 à chaque coup d'horloge.



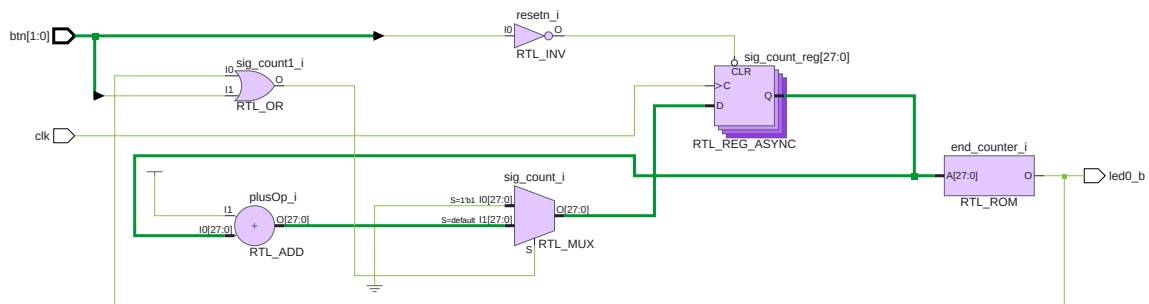
¹ Un *generic* de l'entité `counter_unit` permet de configurer K .

Question 3

Ajoutez une condition pour que le compteur soit remis à 0 lorsqu'il a atteint la valeur souhaitée.



L'analyse RTL Vivado confirme que l'implémentation de l'entité counter_unit représente bien la schématique présentée Question 3.



On remarquera aussi que, lorsque le generic `cfg_led_flip_flop` a la valeur `false` (défaut) le compilateur Vivado optimise le *statement*

```
led0_b <= sig_led when cfg_led_flip_flop else end_counter;
```

en

```
led0_b <= end_counter;
```

Question 4

Listez les signaux d'entrée, de sortie et les signaux internes de votre architecture.

A ce stade l'interface de l'entité `counter_unit` :

- définit un unique port d'entrée, l'horloge `clk`
- ne définit aucun réel port de sortie

Elle définit cependant des signaux représentant :

- l'état interne du compteur (*`sig_count`* et *`end_counter`*)
- la commande CLR du compteur (`resetn`)

Rem : Une telle entité sans port de sortie sera typiquement instanciée lors de simulations permettant de tester l'évolution de son état interne .

Question 5

Ecrivez à présent le compteur en VHDL en suivant le schéma RTL, faites attention de bien faire correspondre les noms des signaux de votre code VHDL avec ceux de votre schéma RTL.

Voir `src/counter.vhd`.

Question 6

Ecrivez un fichier de testbench pour tester votre design.

Voir `src/tb_counter.vhd`

Rem : L'interface de l'entité `counter_unit` définit une constante *generic* permettant de configurer la périodicité K du compteur, exprimée en nombre de fronts montants de l'horloge.

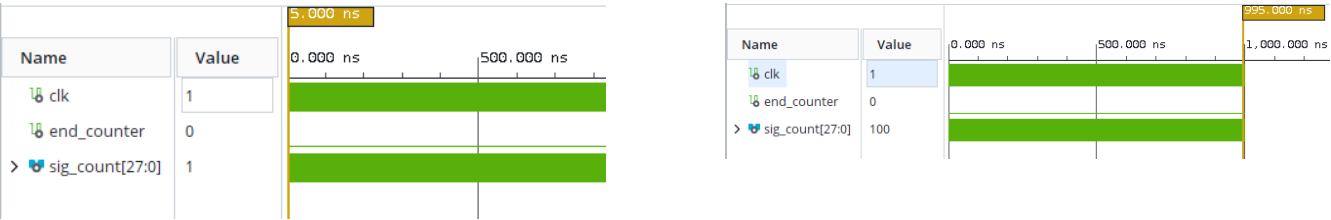
Pour une d'horloge de période $T=10\text{ ns}$, le temps simulé sera alors $\Delta t=KT$, soit exprimé en nanosecondes $\Delta t=10\times K$. La plupart des exemples qui suivent configurent la simulation avec $K=2000$, soit une périodicité du compteur $\Delta t=10\times 2000\text{ ns}$ de 20 us.

Question 7

Lancez une simulation. Que devez-vous observer sur votre chronogramme pour vérifier que votre design est valide ?

En exécutant la simulation `tb_counter.vhd` pendant 1 ns (défaut) nous vérifions :

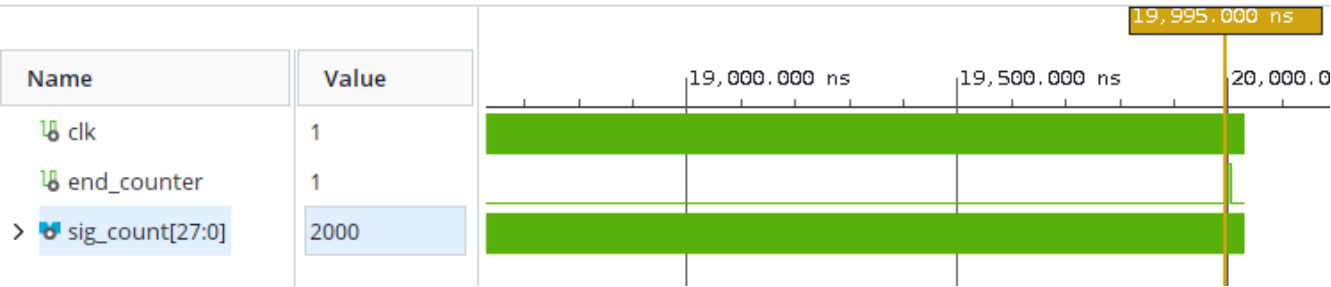
- Le compteur est incrémenté une première fois après 5 ns, c'est à dire après $\frac{T}{2}$:
l'horloge étant initialement à l'état LOW, le premier front montant apparaît après une demi-période.
- À l'instant $t=995\text{ ns}$, c'est à dire 99 périodes plus tard, nous mesurons bien 100 fronts montants



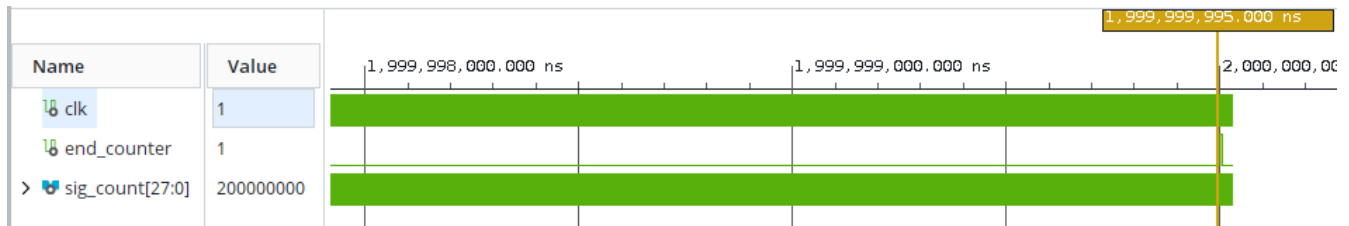
En exécutant la simulation avec $K=2000$, pendant 20.03 us, nous vérifions :

- Après 19995 ns, c'est à dire après une demi-période plus 1999 périodes, le compteur mesure bien 2000 fronts montants, et le signal interne `end_counter` montre que nous avons atteint le nombre maximum de périodes.
- Le compteur est remis à zéro sur le front montant suivant.

t (ns)	clk	<i>end_counter</i>	<i>sig_count</i>
t_0	0	0	0
5	1	0	1
15	1	0	2
...	...	...	...
19985	1	0	1999
19995	1	1	2000
20005	1	0	0
20015	1	0	1
20025	1	0	2



En exécuter la simulation² avec $K=2.10^8$, pendant 20000000.03 us, nous vérifions de même qu'après 1999999.995 us, c'est à dire après une demi-période plus 199999999 périodes, le compteur mesure bien 2.10^8 fronts montants.



Rem : exécuter cette simulation avec Vivado 26.01 demandera 15 minutes (Intel i7 @2.6GHz) ... et l'espace disque nécessaires à la capture des 2×2.10^8 évènements, soit plus de 10 GB³.

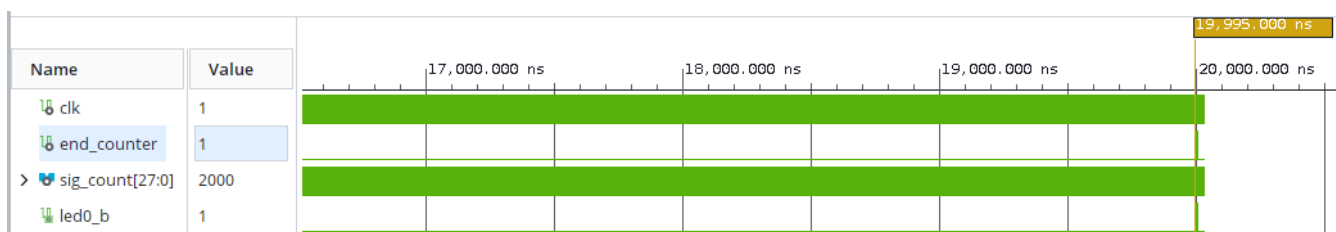
Question 8

Associez une LED avec le signal de teste d'arrêt du compteur. Pour cela, il faudra ajouter une sortie et la relier à une broche d'une LED dans le fichier de contrainte (.xdc). La LED sera alors allumée pendant seulement un coup d'horloge.

La LED est représentée par le port de sortie led0_b du compteur :

- Voir src/counter_unit.vhd et src/Cora-Z7-10-Master.xdc
- Voir schéma RTL Question 9.

Exécuter cette simulation avec $K=2000$, pendant 20.03 us, nous permet de vérifier que la LED est bien ON lorsque le signal interne *end_counter* est HIGH.



Question 9

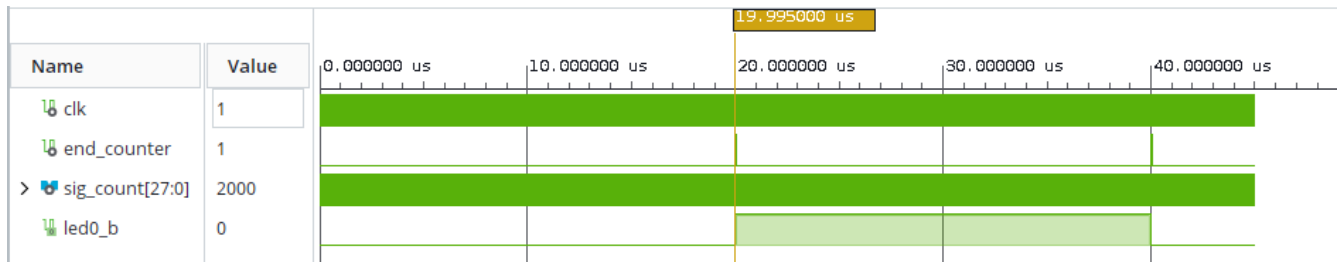
Modifiez le schéma RTL du compteur pour ajouter une remise à 0 lorsqu'un signal restart est à 1. Ajoutez la logique nécessaire pour que la LED clignote telle que : allumée 2s, éteinte 2s.

Le principe est d'inverser l'état du port de sortie led0_b sur chaque évènement *end_counter*, dont la périodicité est de 2 s. La périodicité de la LED serait alors 4 seconds.

² Configurer K dans tb_counter.vhd.

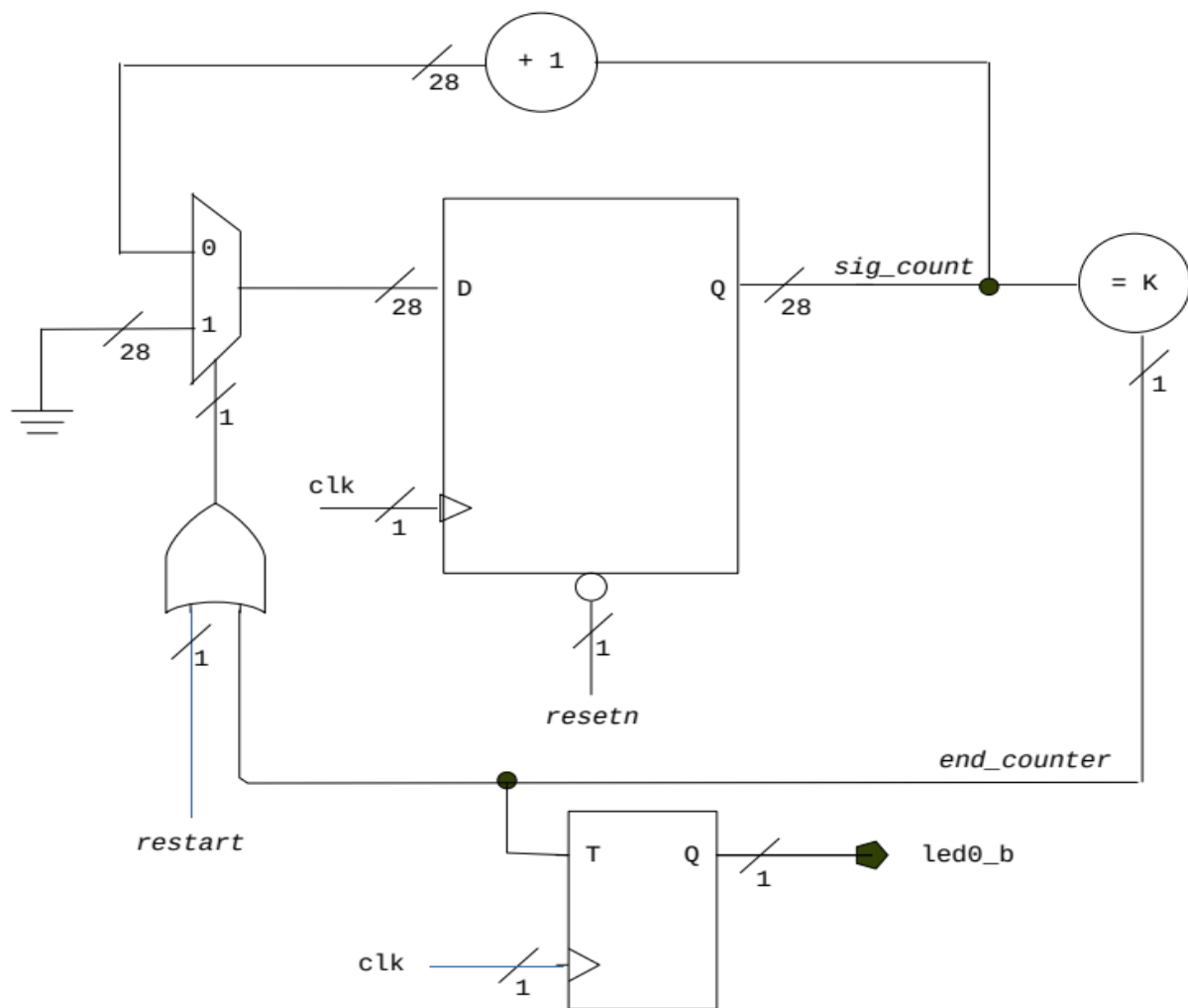
³ Sur Linux, augmenter la capacité du système de fichiers temporaires (tmpfs), par exemple `mount -o remount,size=32G /tmp`.

Par exemple en exécutant la simulation⁴ avec $K=2000$, pendant 45 us, on peut vérifier que la LED change d'état (ON) après la première période compteur (à 20 us), puis une nouvelle fois après une deuxième période compteur (à 40 us).



Comparé à la question 3, le schéma RTL introduit :

- Un Flip Flop préservant le bit d'état de la LED
- Une disjonction (porte OR) ajoutant la signal *restart* au signal *end_counter* pour déterminer si le compteur doit être remis à zéro



4 Configurer `cfg_led_flip_flop` dans `tb_counter.vhd`.

Question 10

Faites les mises à jour nécessaires sur le code VHDL pour correspondre au nouveau schéma.

Voir `src/counter_unit.vhd`.

Question 11

Associez la nouvelle entrée *restart* à un bouton.

Voir `src/counter_unit.vhd`.

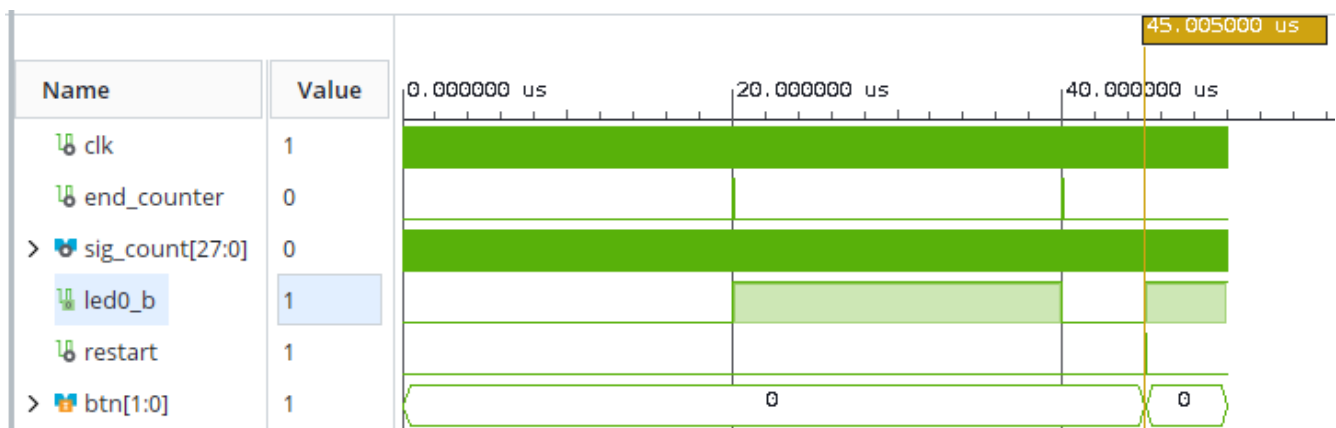
Question 12

Mettez à jour votre testbench puis vérifiez votre design avec une simulation. Quels sont les signaux que vous devez observer ?

Voir `src/tb_counter_unit.vhd`.

Pour simuler un appui sur le bouton *restart* après 45 us, configurer le *generic t_restart* dans `tb_counter.vhd`. Exécuter ensuite la simulation pendant 50 us :

- Après 20 us, la valeur maximum du compteur est atteinte
- Le compteur est remis à zéro, le port de sortie LED passe à l'état HIGH.
- Le compteur atteint à nouveau son maximum autour de 40 us: remise à zéro du compteur, changement d'état de la LED, la simulation entre dans la troisième période compteur.
- À l'instant $t = 40\text{ us}$, le bouton connecté au signal *restart* déclenche une remise à zéro du compteur (et un changement d'état de la LED).



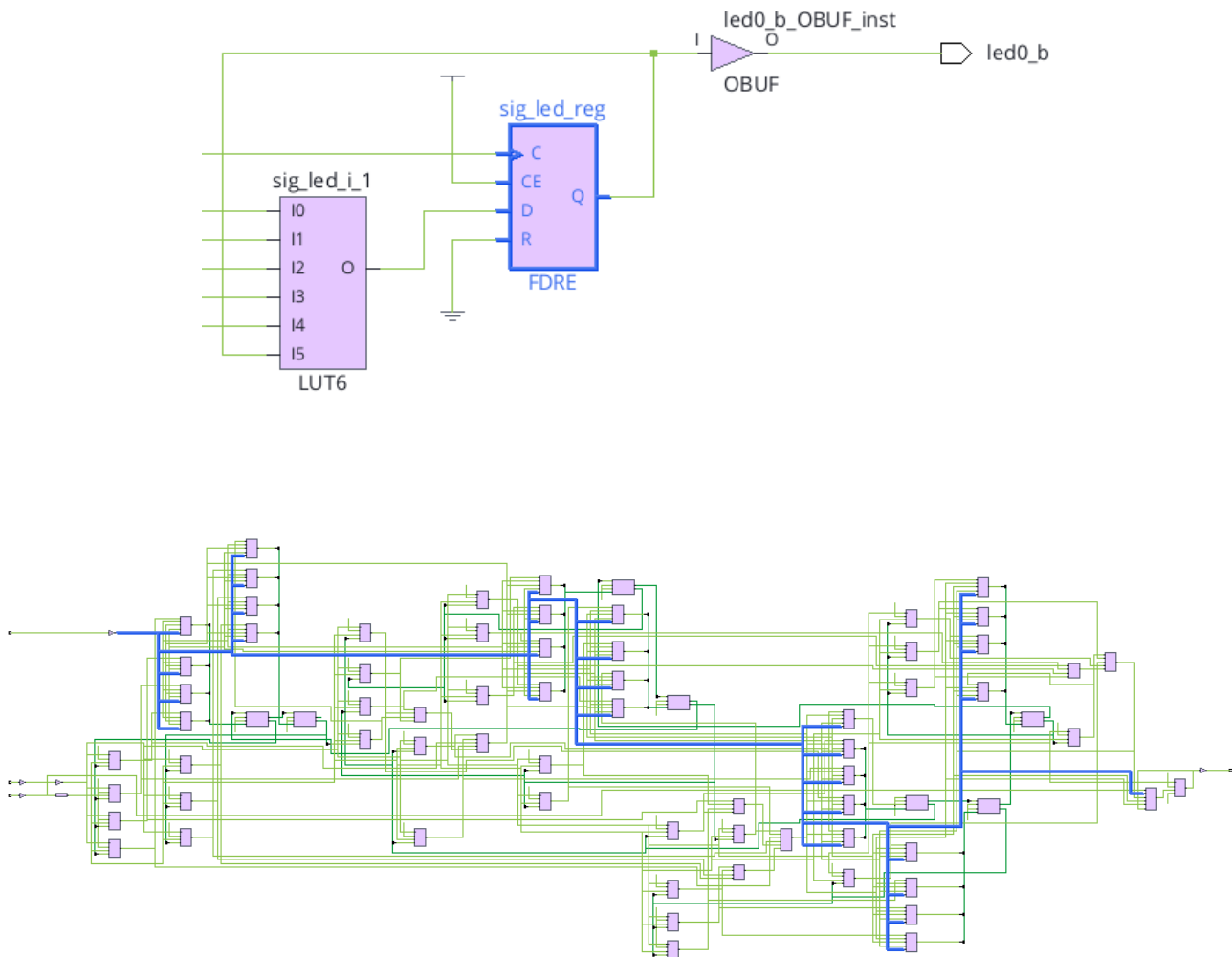
Question 13

Exécutez la synthèse puis ouvrez la schématique. Identifiez sur la schématique les différents éléments de votre architecture RTL.

La schématique complète se trouve dans le fichier `rpt/schematic.pdf`.

On remarquera par exemple :

- la connexion du bouton restart à chaque LUT déterminant l'état d'un bit `sig_count[i]`
- la bascule déterminant et maintenant l'état du bit de sortie `led0_b`



Question 14

Ouvrez le rapport de synthèse et relevez les ressources utilisées. Comparez vos résultats avec les résultats attendu selon votre architecture RTL.

Le rapport de synthèse rpt/utilization_report.txt résume les ressources utilisées par l'entité counter_unit.

On vérifie notamment la présence de 29 registres Flip Flop (28 pour sig_count, plus un pour piloter la LED) .

Question 15

Ouvrez le Set Up Debug. Placez des sondes sur les signaux à observer que vous avez défini à la question 12.

La version 26.04 de Vivado, utilisée avec une licence d'évaluation, ne permet pas la mise en œuvre de sondes ILA :

[Vivado 12-29205] 'create_debug_core' tcl command is not supported. Your current selected license is BASIC. This license doesn't cover the feature you selected. See the Vivado Licensing page for what features are supported in each tier and how to get the right license based on your requirements.

Question 16

Lancez l'implémentation puis étudiez le rapport de timing (vérifiez les violations de set up et de hold et identifiez le chemin critique).

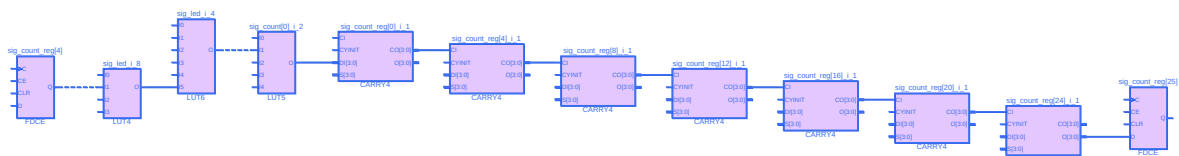
Le rapport de timing attire l'attention sur le défaut de délais en entrée (bouton btn[0] et signal resetn) et sortie (la LED led0_b) : cela est causé est par l'absence de contraintes sur ces signaux, et nous pouvons ici ignorer ces messages.

WNS 5.180 ns
TNS 0.000 ns

WHS 0.281 ns
THS 0.000 ns

WPWS 4.500 ns
TPWS 0.000 ns

Le chemin critique (WNS) propage le bit de retenue de l'additionneur sur 6 registres de sig_count.



Question 17

Générez le *bitstream* pour observer le système sur carte et le résultat de la ILA.

Afin de pouvoir générer le *bitstream*, nous devons configurer le bouton connecté au signal `resetn` en mode Pull-UP :

```
set_property -dict { PACKAGE_PIN D19    IOSTANDARD LVCMOS33 } [get_ports { btn[1] }];  
set_property PULLUP true [get_ports {btn[1]}]
```

Remarquons que le *bitstream* représentant ce simple compteur et deux boutons et une LED pèse 2 MB. Il sera intéressant de comparer avec des systèmes plus complexes.

Le *device* ainsi programmé semble effectivement se comporter comme attendu : la LED clignote avec une période *perceptible* de 4s. Il ne s'agit pas d'une validation du système, mais d'une information qui pourrait apparaître dans la section Inspection d'une document IDAT.